

SELF – LEARNING (DBT)

❖ MongoDB Lab Exercises

1. Practice CRUD operations on Mongo.

1. Create two collections -- emp and dept.

```
test> db.createCollection("emp")
| db.createCollection("dept")
{ ok: 1 }
test>
| db.createCollection("dept")
{ ok: 1 }
test> show collections
dept
emp
```

2. Insert the emp and dept records.

```
test> db.emp.insertMany([
|   {empno: 7369, ename: "SMITH", job: "CLERK", mgr: 7902, sal: 800, comm: null, deptno: 20},
|   {empno: 7499, ename: "ALLEN", job: "SALESMAN", mgr: 7698, sal: 1600, comm: 300, deptno: 30},
|   {empno: 7521, ename: "WARD", job: "SALESMAN", mgr: 7698, sal: 1250, comm: 500, deptno: 30},
|   {empno: 7566, ename: "JONES", job: "MANAGER", mgr: 7839, sal: 2975, comm: null, deptno: 20},
|   {empno: 7654, ename: "MARTIN", job: "SALESMAN", mgr: 7698, sal: 1250, comm: 1400, deptno: 30},
|   {empno: 7698, ename: "BLAKE", job: "MANAGER", mgr: 7839, sal: 2850, comm: null, deptno: 30},
|   {empno: 7782, ename: "CLARK", job: "MANAGER", mgr: 7839, sal: 2450, comm: null, deptno: 10},
|   {empno: 7788, ename: "SCOTT", job: "ANALYST", mgr: 7566, sal: 3000, comm: null, deptno: 20},
|   {empno: 7839, ename: "KING", job: "PRESIDENT", mgr: null, sal: 5000, comm: null, deptno: 10},
|   {empno: 7844, ename: "TURNER", job: "SALESMAN", mgr: 7698, sal: 1500, comm: 0, deptno: 30},
|   {empno: 7876, ename: "ADAMS", job: "CLERK", mgr: 7788, sal: 1100, comm: null, deptno: 20},
|   {empno: 7900, ename: "JAMES", job: "CLERK", mgr: 7698, sal: 950, comm: null, deptno: 30},
|   {empno: 7902, ename: "FORD", job: "ANALYST", mgr: 7566, sal: 3000, comm: null, deptno: 20},
|   {empno: 7934, ename: "MILLER", job: "CLERK", mgr: 7782, sal: 1300, comm: null, deptno: 10}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aa93572b34f2f3a1a7c2907'),
    '1': ObjectId('6aa93572b34f2f3a1a7c2908'),
    '2': ObjectId('6aa93572b34f2f3a1a7c2909'),
    '3': ObjectId('6aa93572b34f2f3a1a7c290a'),
    '4': ObjectId('6aa93572b34f2f3a1a7c290b'),
    '5': ObjectId('6aa93572b34f2f3a1a7c290c'),
    '6': ObjectId('6aa93572b34f2f3a1a7c290d'),
    '7': ObjectId('6aa93572b34f2f3a1a7c290e'),
    '8': ObjectId('6aa93572b34f2f3a1a7c290f'),
    '9': ObjectId('6aa93572b34f2f3a1a7c2910'),
    '10': ObjectId('6aa93572b34f2f3a1a7c2911'),
    '11': ObjectId('6aa93572b34f2f3a1a7c2912'),
    '12': ObjectId('6aa93572b34f2f3a1a7c2913'),
    '13': ObjectId('6aa93572b34f2f3a1a7c2914')
  }
}
```

```
test> db.dept.insertMany([
|   {deptno: 10, dname: "ACCOUNTING", loc: "NEW YORK"},
|   {deptno: 20, dname: "RESEARCH", loc: "DALLAS"},
|   {deptno: 30, dname: "SALES", loc: "CHICAGO"},
|   {deptno: 40, dname: "OPERATIONS", loc: "BOSTON"}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aa9357eb34f2f3a1a7c2915'),
    '1': ObjectId('6aa9357eb34f2f3a1a7c2916'),
    '2': ObjectId('6aa9357eb34f2f3a1a7c2917'),
    '3': ObjectId('6aa9357eb34f2f3a1a7c2918')
  }
}
```

3. Increase salary of dept 10 emps by 100.

```
test> db.emp.updateMany(
|   {deptno: 10},
|   {$inc: {sal: 100}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000.

```
test> db.emp.find(
|   {sal: {$gt: 1000}},
|   {_id: 0, ename: 1, job: 1, deptno: 1, sal: 1}
| ).sort(
|   {deptno: 1, job: 1, sal: -1}
| )
[
  { ename: 'MILLER', job: 'CLERK', sal: 1400, deptno: 10 },
  { ename: 'CLARK', job: 'MANAGER', sal: 2550, deptno: 10 },
  { ename: 'KING', job: 'PRESIDENT', sal: 5100, deptno: 10 },
  { ename: 'SCOTT', job: 'ANALYST', sal: 3000, deptno: 20 },
  { ename: 'FORD', job: 'ANALYST', sal: 3000, deptno: 20 },
  { ename: 'ADAMS', job: 'CLERK', sal: 1100, deptno: 20 },
  { ename: 'JONES', job: 'MANAGER', sal: 2975, deptno: 20 },
  { ename: 'BLAKE', job: 'MANAGER', sal: 2850, deptno: 30 },
  { ename: 'ALLEN', job: 'SALESMAN', sal: 1600, deptno: 30 },
  { ename: 'TURNER', job: 'SALESMAN', sal: 1500, deptno: 30 },
  { ename: 'WARD', job: 'SALESMAN', sal: 1250, deptno: 30 },
  { ename: 'MARTIN', job: 'SALESMAN', sal: 1250, deptno: 30 }
]
```

5. Delete the dept with id 40.

```
test> db.dept.deleteOne(
|   {deptno: 40}
| )
{ acknowledged: true, deletedCount: 1 }
```

2. Learn grouping operations using Mongo aggregation pipeline

1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT.

```
{
  _id: 'ANALYST',
  total_sal: 6000,
  max_sal: 3000,
  min_sal: 3000,
  avg_sal: 3000
},
{
  _id: 'MANAGER',
  total_sal: 8375,
  max_sal: 2975,
  min_sal: 2550,
  avg_sal: 2791.6666666666665
},
{
  _id: 'CLERK',
  total_sal: 4250,
  max_sal: 1400,
  min_sal: 800,
  avg_sal: 1062.5
},
{
  _id: 'SALESMAN',
  total_sal: 5600,
  max_sal: 1600,
  min_sal: 1250,
  avg_sal: 1400
}
```

2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

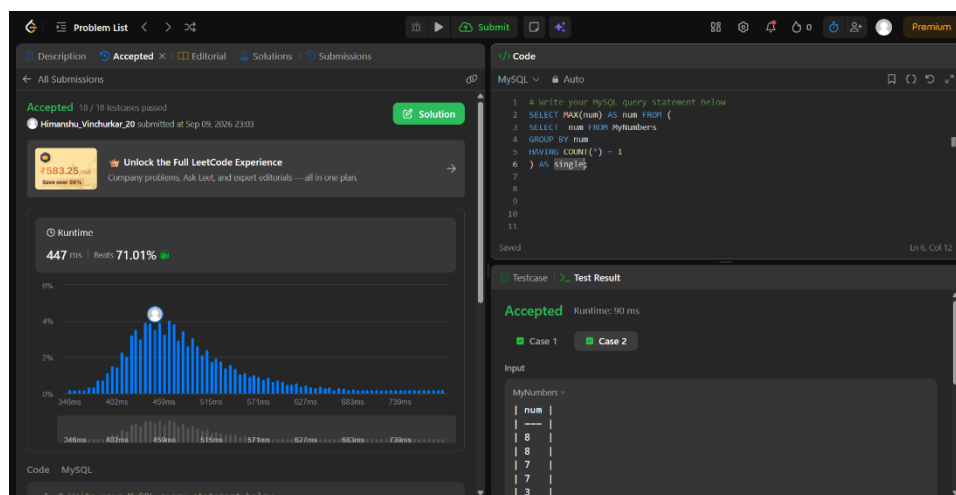
```
test> db.emp.aggregate([
  {
    $lookup: {
      from: "dept",
      localField: "deptno",
      foreignField: "deptno",
      as: "dept"
    }
  },
  {
    $unwind: "$dept"
  },
  {
    $project: {
      _id: 0,
      ename: 1,
      dname: "$dept.dname",
      income: { $add: ["$sal", "$comm"] }
    }
  },
  {
    $sort: {
      income: 1
    }
  }
])
[
  { ename: 'SMITH', dname: 'RESEARCH', income: null },
  { ename: 'JONES', dname: 'RESEARCH', income: null },
  { ename: 'BLAKE', dname: 'SALES', income: null },
  { ename: 'CLARK', dname: 'ACCOUNTING', income: null },
  { ename: 'SCOTT', dname: 'RESEARCH', income: null },
  { ename: 'KING', dname: 'ACCOUNTING', income: null },
  { ename: 'ADAMS', dname: 'RESEARCH', income: null },
  { ename: 'JAMES', dname: 'SALES', income: null },
  { ename: 'FORD', dname: 'RESEARCH', income: null },
  { ename: 'MILLER', dname: 'ACCOUNTING', income: null },
  { ename: 'TURNER', dname: 'SALES', income: 1500 },
  { ename: 'WARD', dname: 'SALES', income: 1750 },
  { ename: 'ALLEN', dname: 'SALES', income: 1900 },
  { ename: 'MARTIN', dname: 'SALES', income: 2650 }
]
test> |
```

❖ GROUP BY Problems :

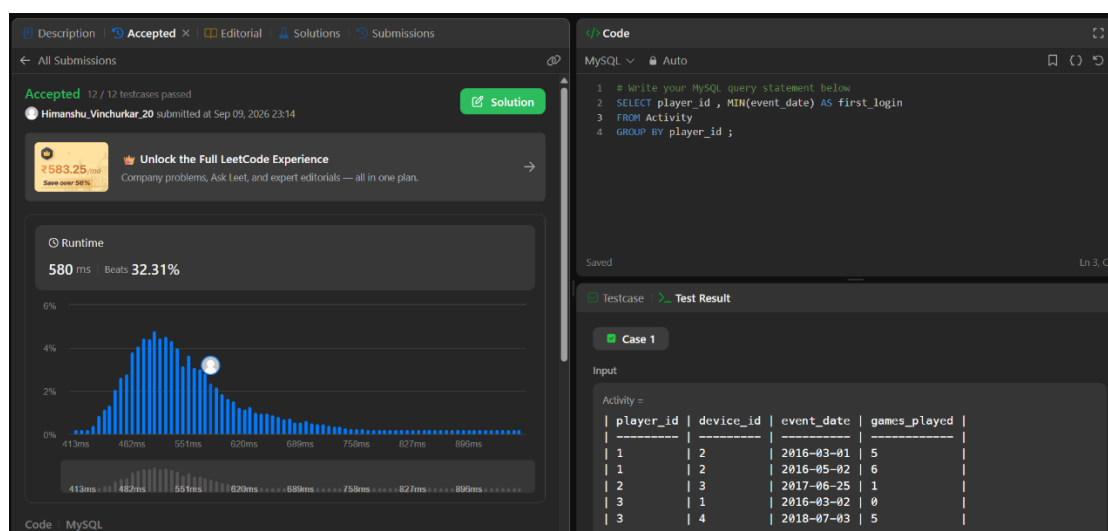
1. 586. Customer Placing the Largest Number of Orders <https://leetcode.com/problems/customer-placing-the-largest-number-of-orders>

The screenshot shows the LeetCode interface for problem 586, "Customer Placing the Largest Number of Orders". The left sidebar contains the problem description, a submission by Himanshu_Vinchurkar_20, and a runtime performance graph. The main area displays the MySQL code used to solve the problem: `SELECT customer_number FROM Orders GROUP BY customer_number ORDER BY COUNT(*) DESC LIMIT 1;`. The test result is "Accepted" with a runtime of 100 ms.

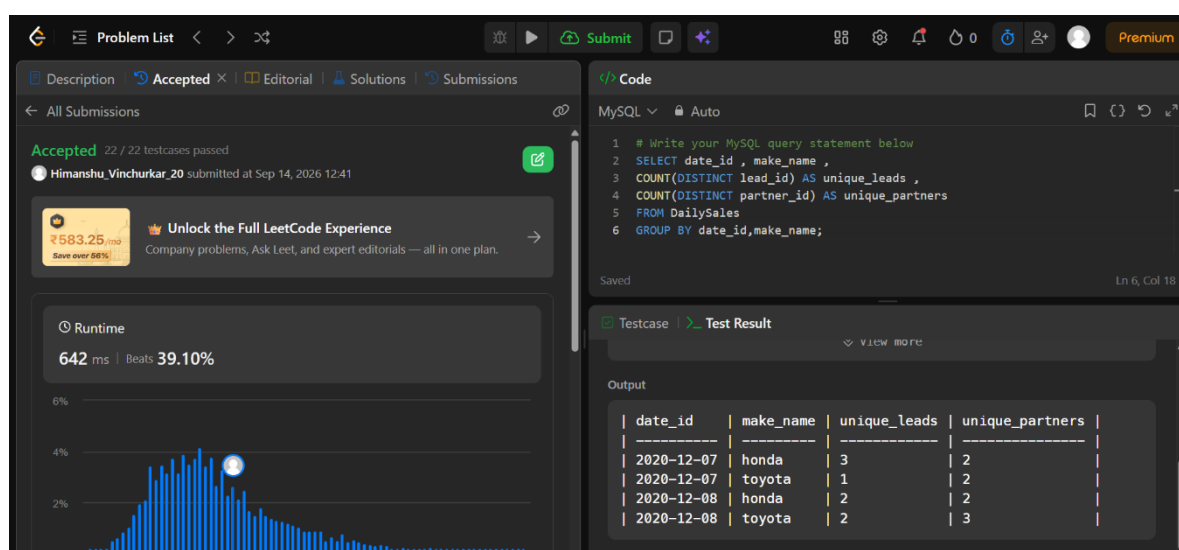
2. 619. Biggest Single Number <https://leetcode.com/problems/biggest-single-number>



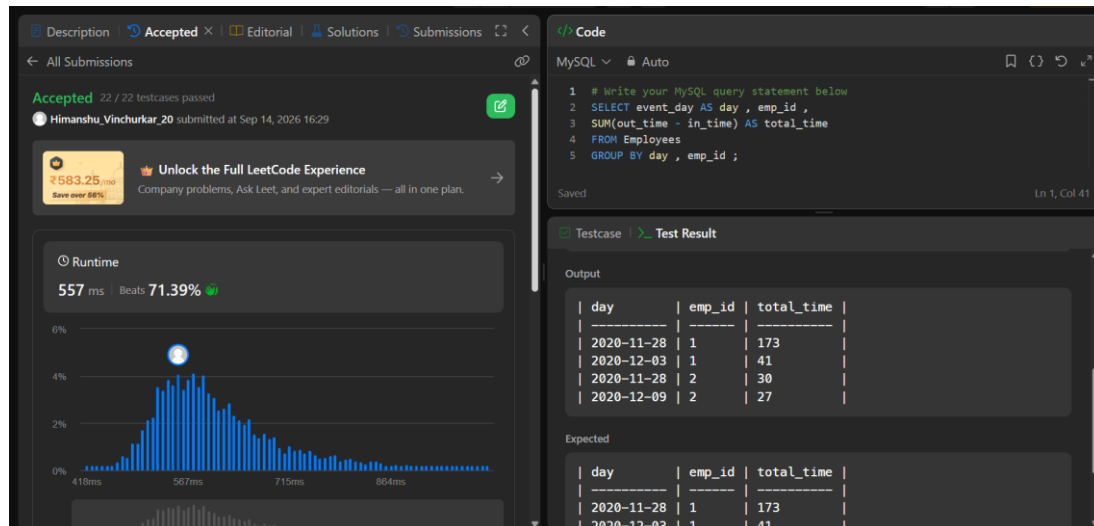
3. 511. Game Play Analysis I <https://leetcode.com/problems/game-play-analysis-i>



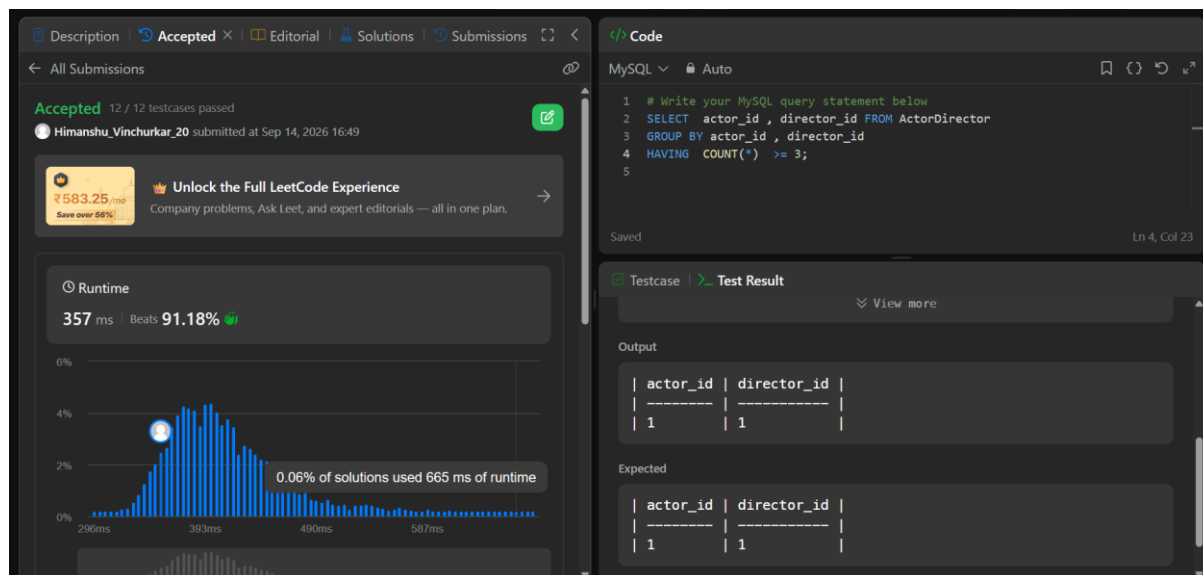
4. 1693. Daily Leads and Partners <https://leetcode.com/problems/daily-leads-and-partners>



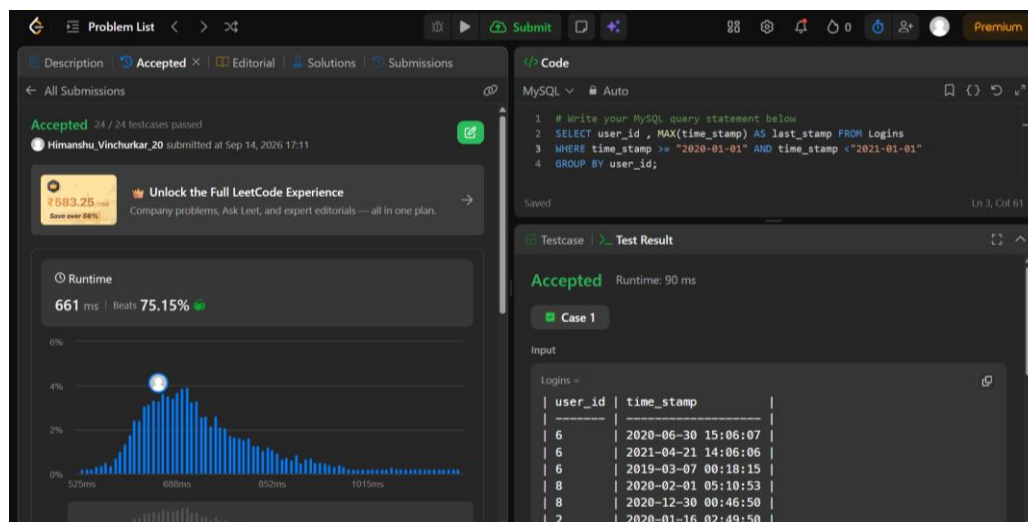
5. 1741. Find Total Time Spent by Each Employee <https://leetcode.com/problems/find-total-time-spent-by-each-employee>



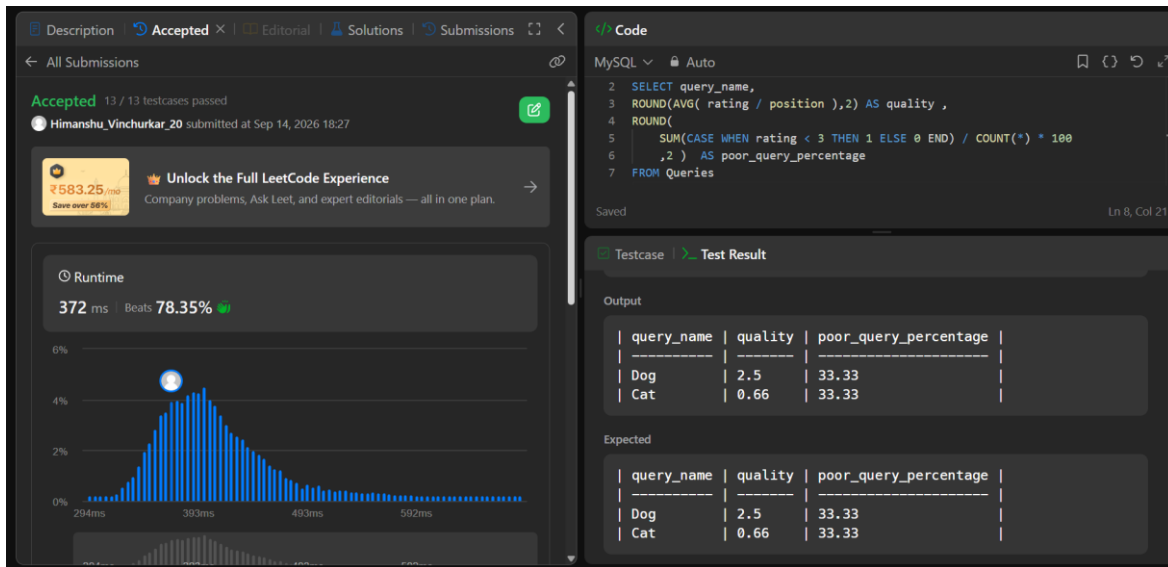
6. 1050. Actors and Directors Who Cooperated at Least Three Times



7. 1890. The Latest Login in 2020 <https://leetcode.com/problems/the-latest-login-in-2020>



8. 1211. Queries Quality and Percentage <https://leetcode.com/problems/queries-quality-and->



Accepted 13 / 13 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 14, 2026 18:27

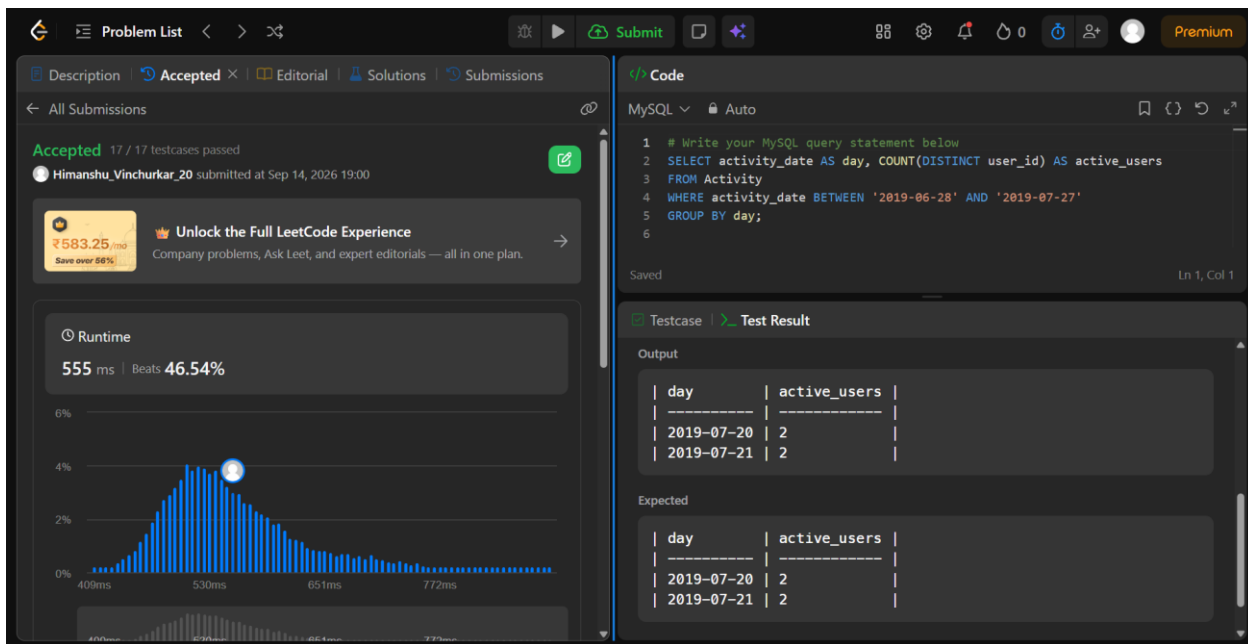
Runtime: 372 ms | Beats 78.35%

```
SELECT query_name,
ROUND(AVG( rating / position ),2) AS quality ,
ROUND(
SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END) / COUNT(*) * 100
,2 ) AS poor_query_percentage
FROM Queries
```

query_name	quality	poor_query_percentage
Dog	2.5	33.33
Cat	0.66	33.33

query_name	quality	poor_query_percentage
Dog	2.5	33.33
Cat	0.66	33.33

9. 1141. User Activity for the Past 30 Days I <https://leetcode.com/problems/user-activity-for-the-past->



Accepted 17 / 17 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 14, 2026 19:00

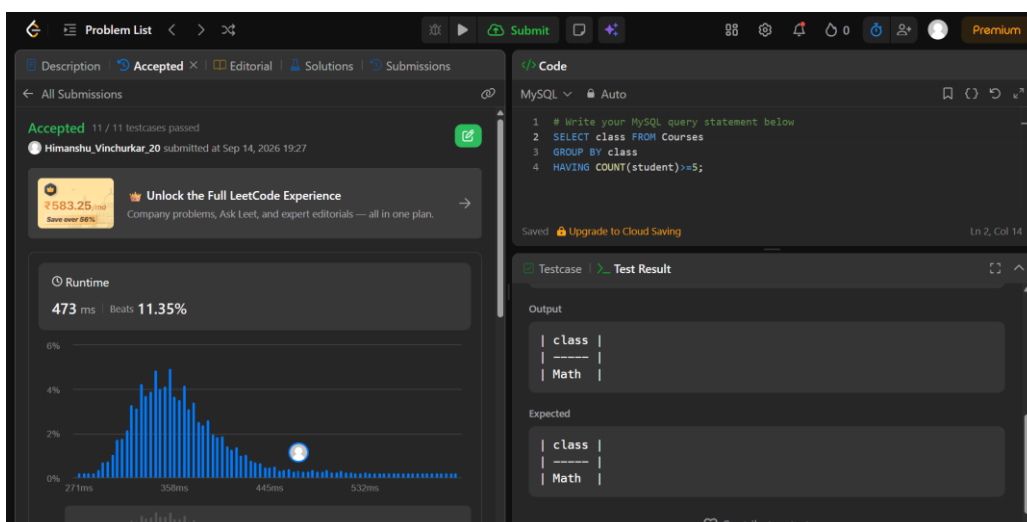
Runtime: 555 ms | Beats 46.54%

```
# Write your MySQL query statement below
SELECT activity_date AS day, COUNT(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN '2019-06-28' AND '2019-07-27'
GROUP BY day;
```

day	active_users
2019-07-20	2
2019-07-21	2

day	active_users
2019-07-20	2
2019-07-21	2

10. 596. Classes With At Least 5 Students <https://leetcode.com/problems/classes-with-at-least-5-students>



Accepted 11 / 11 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 14, 2026 19:27

Runtime: 473 ms | Beats 11.35%

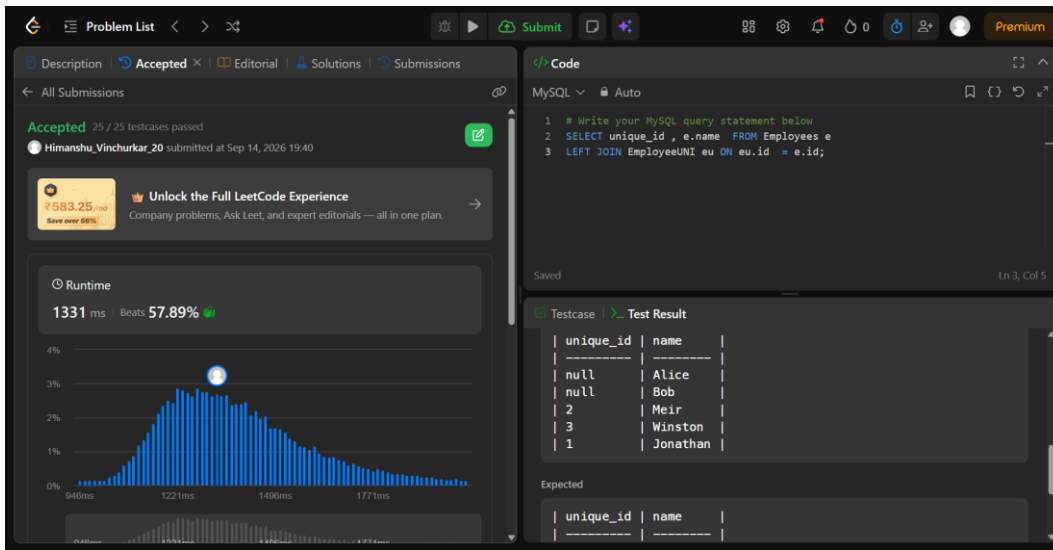
```
# Write your MySQL query statement below
SELECT class FROM Courses
GROUP BY class
HAVING COUNT(student)>=5;
```

class
Math

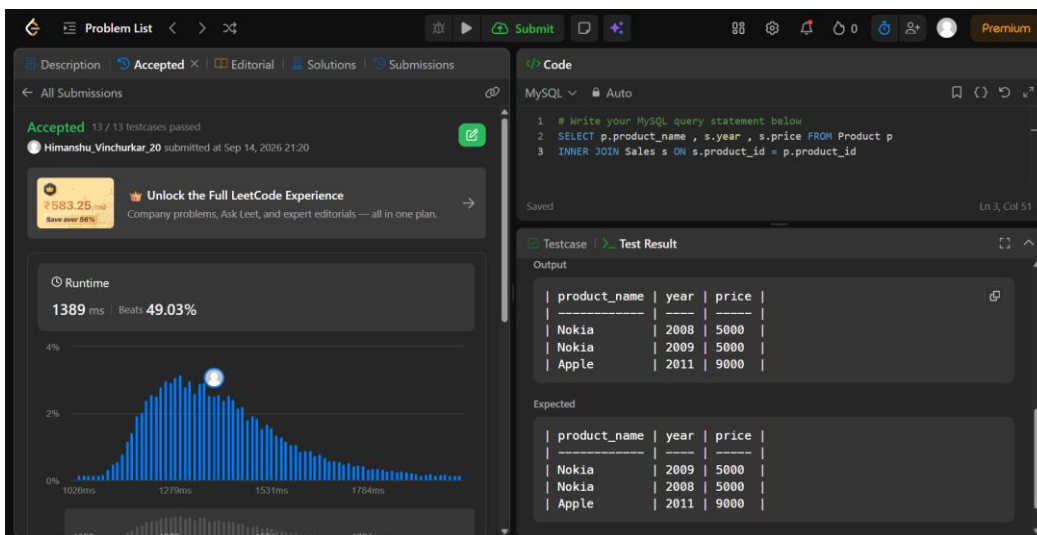
class
Math

❖ JOINS Problems :

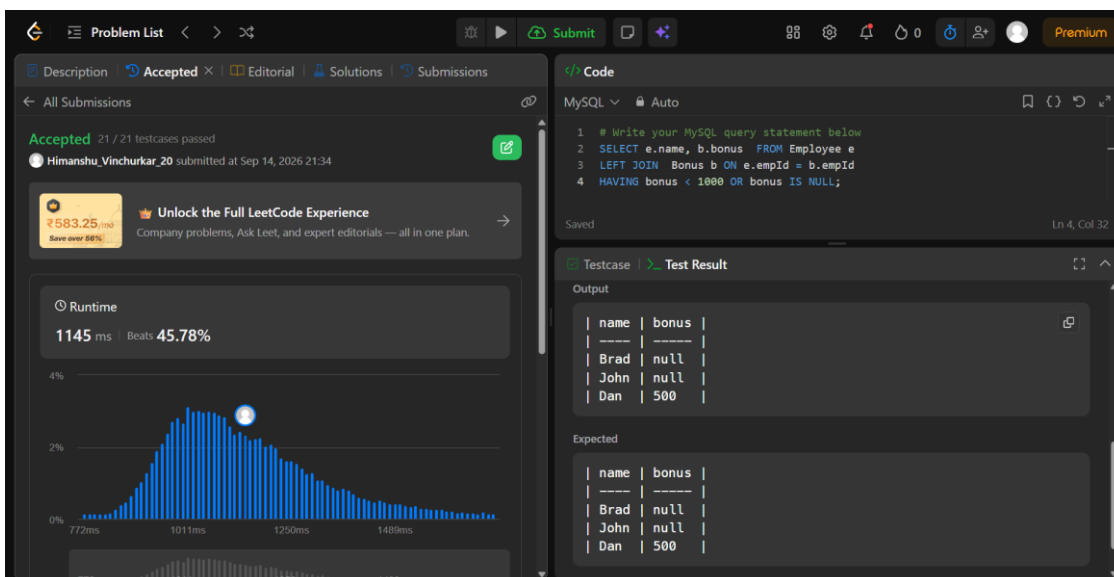
1. Replace Employee ID With The Unique Identifier <https://leetcode.com/problems/replace-employee-id-with-the-unique-identifier>



2. Product Sales Analysis I <https://leetcode.com/problems/product-sales-analysis-i>



3. Employee Bonus <https://leetcode.com/problems/employee-bonus>



4. Students and Examinations <https://leetcode.com/problems/students-and-examinations>

The screenshot shows the LeetCode interface for the 'Students and Examinations' problem. The submission is accepted, with 15/15 testcases passed. The runtime is 950 ms, beating 93.20% of solutions. A bar chart shows the runtime distribution. The code is a MySQL query that joins the Students, Subjects, and Examinations tables to count the number of exams attended by each student for each subject.

```
1 # Write your MySQL query statement below
2 SELECT su.student_id, su.student_name, sb.subject_name,
3 COUNT(e.subject_name) AS attended_exams
4 FROM Students su
5 CROSS JOIN Subjects sb
6 LEFT JOIN Examinations e ON e.student_id = su.student_id AND e.subject_name = sb.subject_name
7 GROUP BY student_id, subject_name
8 ORDER BY student_id, subject_name;
```

student_id	student_name	subject_name	attended_exams
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Physics	0
2	Bob	Programming	1

5. Managers With At Least 5 Direct Reports <https://leetcode.com/problems/managers-with-at-least-5-direct-reports>

The screenshot shows the LeetCode interface for the 'Managers With At Least 5 Direct Reports' problem. The submission is accepted, with 11/11 testcases passed. The runtime is 374 ms, beating 61.55% of solutions. A bar chart shows the runtime distribution. The code is a MySQL query that joins the Employee table with itself to find managers who have at least 5 direct reports.

```
1 # Write your MySQL query statement below
2 SELECT e.name FROM Employee e
3 INNER JOIN Employee t ON t.managerId = e.id
4 GROUP BY t.managerId
5 HAVING COUNT(t.id) >= 5;
```

name
John

Expected:

name
John

6. Asian Population <https://www.hackerrank.com/challenges/asian-population>

The screenshot shows the HackerRank interface for the 'Asian Population' challenge. A green banner at the top says 'Congratulations' and 'You solved this challenge. Would you like to challenge your friends?'. Below the banner, the test case details are shown. The compiler message is 'Success'. The input (stdin) is '1'. The expected output is '27828484'.

Test case 0

Compiler Message

Success

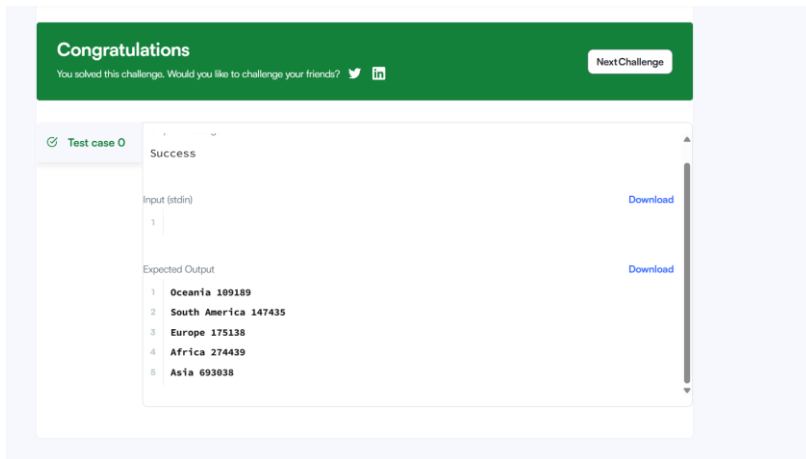
Input (stdin)

1

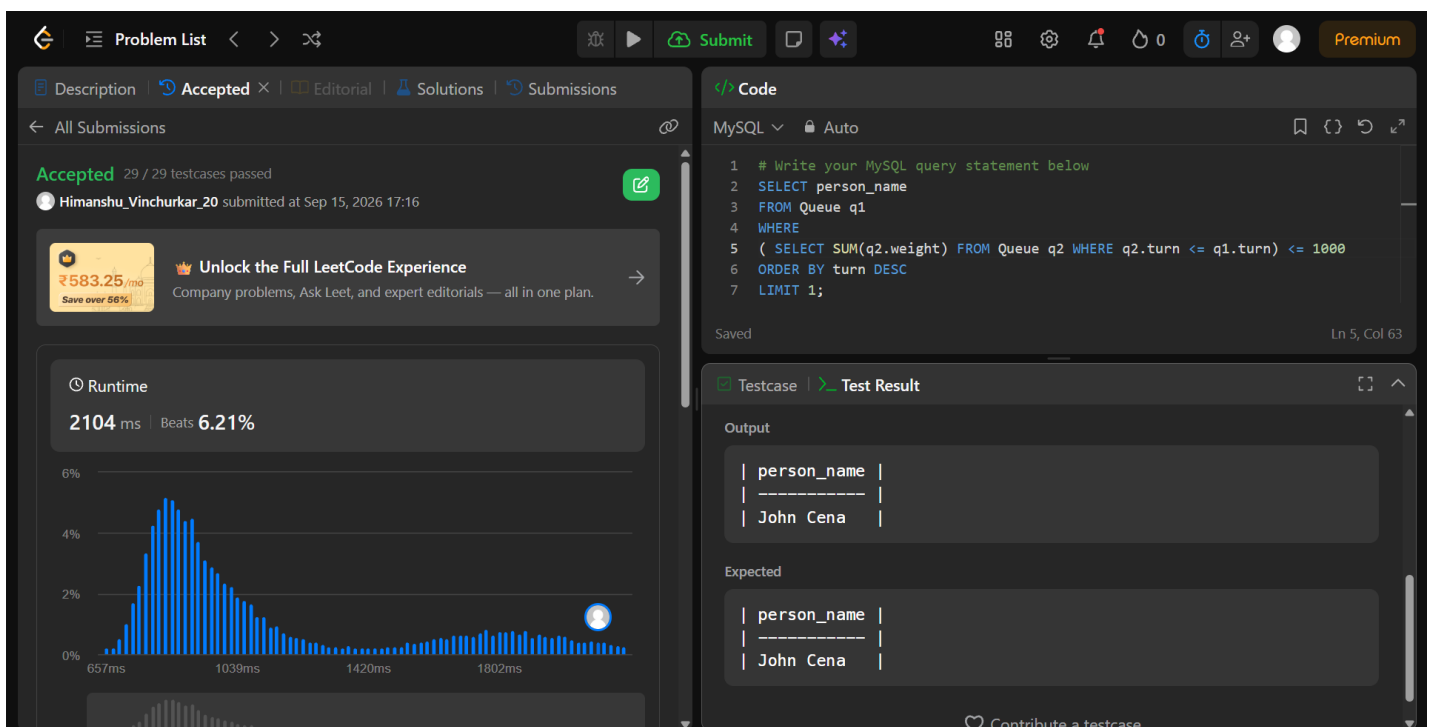
Expected Output

1 27828484

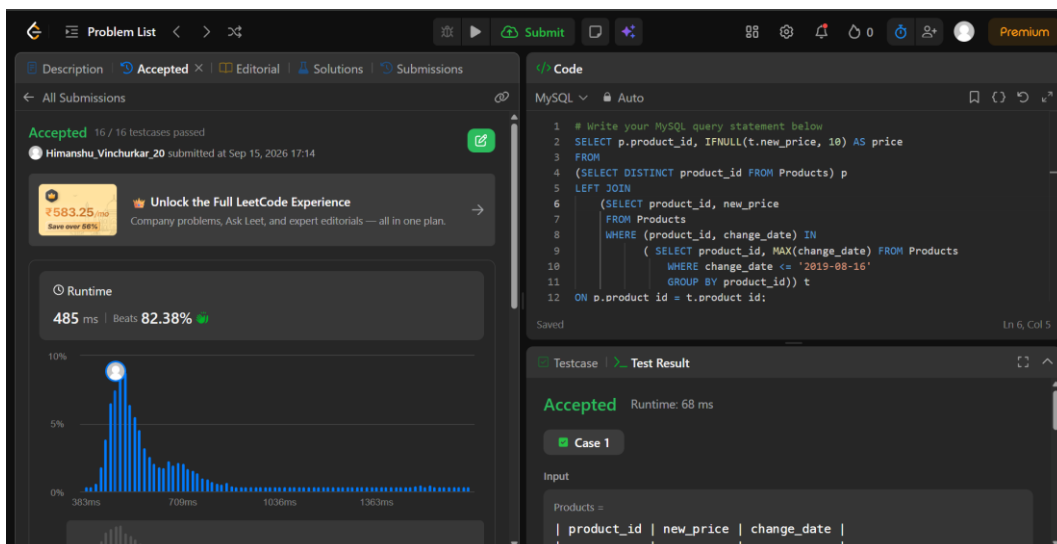
7. Average Population of Each Continent <https://www.hackerrank.com/challenges/average-population-of-each-continent>



8. Last Person to Fit in the Bus <https://leetcode.com/problems/last-person-to-fit-in-the-bus>

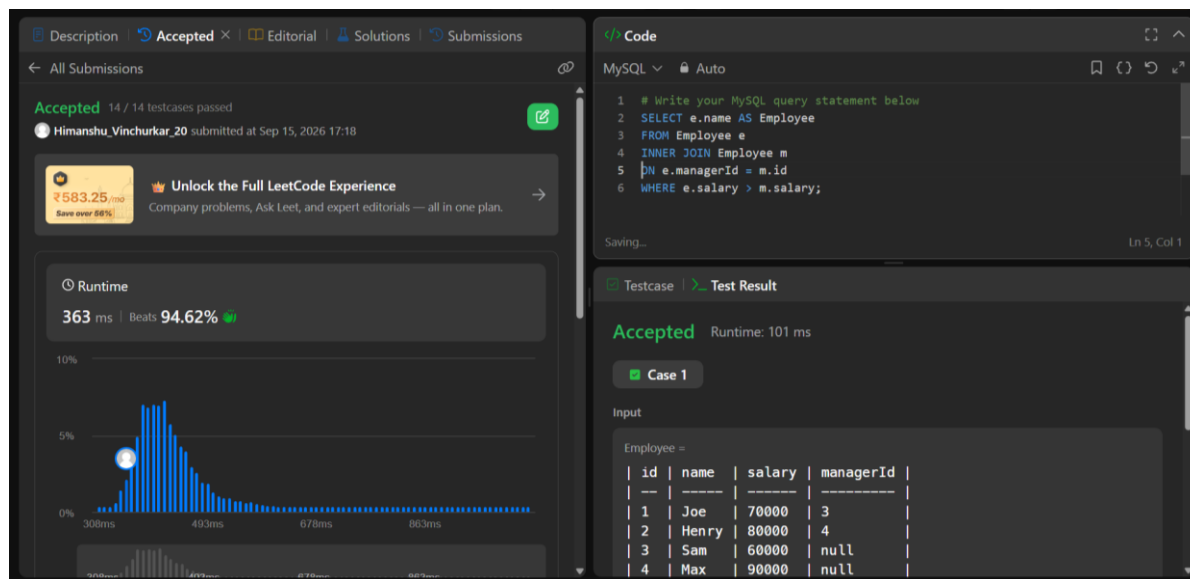


9. Product Price at a Given Date <https://leetcode.com/problems/product-price-at-a-given-date>



Subquery Problems

1. 181. Employees Earning More Than Their Managers <https://leetcode.com/problems/employees-earning-more-than-their-managers/>



Accepted 14 / 14 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:18

₹583.25/mo Save over 58%
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
363 ms | Beats 94.62%

10%
5%
0%
300ms 400ms 678ms 863ms

Code
MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT e.name AS Employee
3 FROM Employee e
4 INNER JOIN Employee m
5 ON e.managerId = m.id
6 WHERE e.salary > m.salary;
```

Saving... Ln 5, Col 1

Testcase | **Test Result**
Accepted Runtime: 101 ms

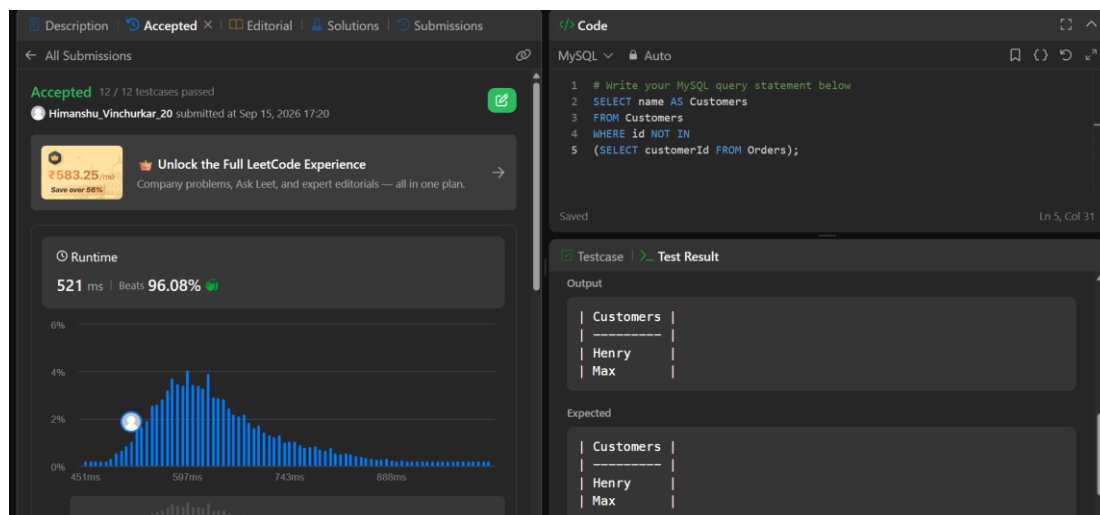
Case 1

Input

Employee =

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	null
4	Max	90000	null

2. 183. Customers Who Never Order <https://leetcode.com/problems/customers-who-never-order/>



Accepted 12 / 12 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:20

₹583.25/mo Save over 58%
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
521 ms | Beats 96.08%

6%
4%
2%
0%
451ms 597ms 743ms 888ms

Code
MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT name AS Customers
3 FROM Customers
4 WHERE id NOT IN
5 (SELECT customerId FROM Orders);
```

Saved Ln 5, Col 31

Testcase | **Test Result**

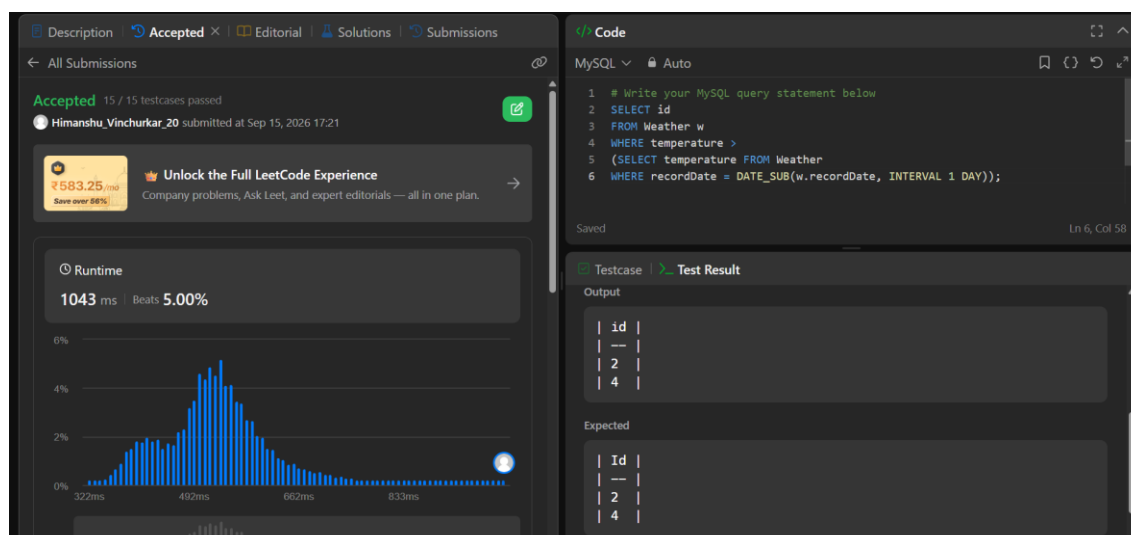
Output

Customers
Henry
Max

Expected

Customers
Henry
Max

3. 197. Rising Temperature <https://leetcode.com/problems/rising-temperature/>



Accepted 15 / 15 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:21

₹583.25/mo Save over 58%
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
1043 ms | Beats 5.00%

6%
4%
2%
0%
322ms 492ms 662ms 833ms

Code
MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT id
3 FROM Weather w
4 WHERE temperature >
5 (SELECT temperature FROM Weather
6 WHERE recordDate = DATE_SUB(w.recordDate, INTERVAL 1 DAY));
```

Saved Ln 6, Col 58

Testcase | **Test Result**

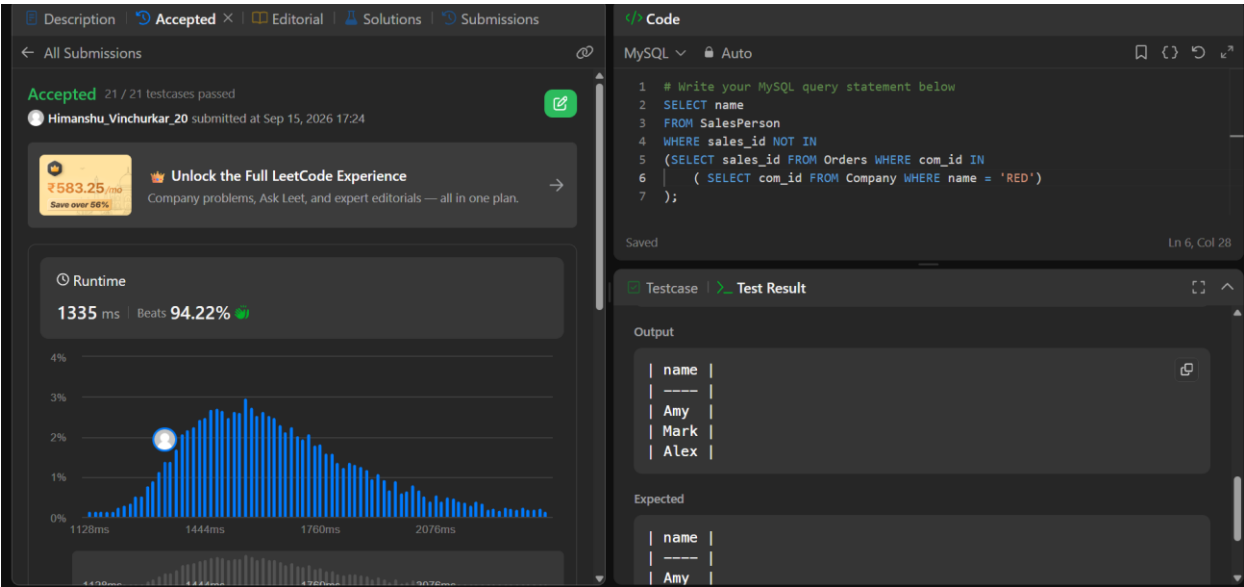
Output

id
2
4

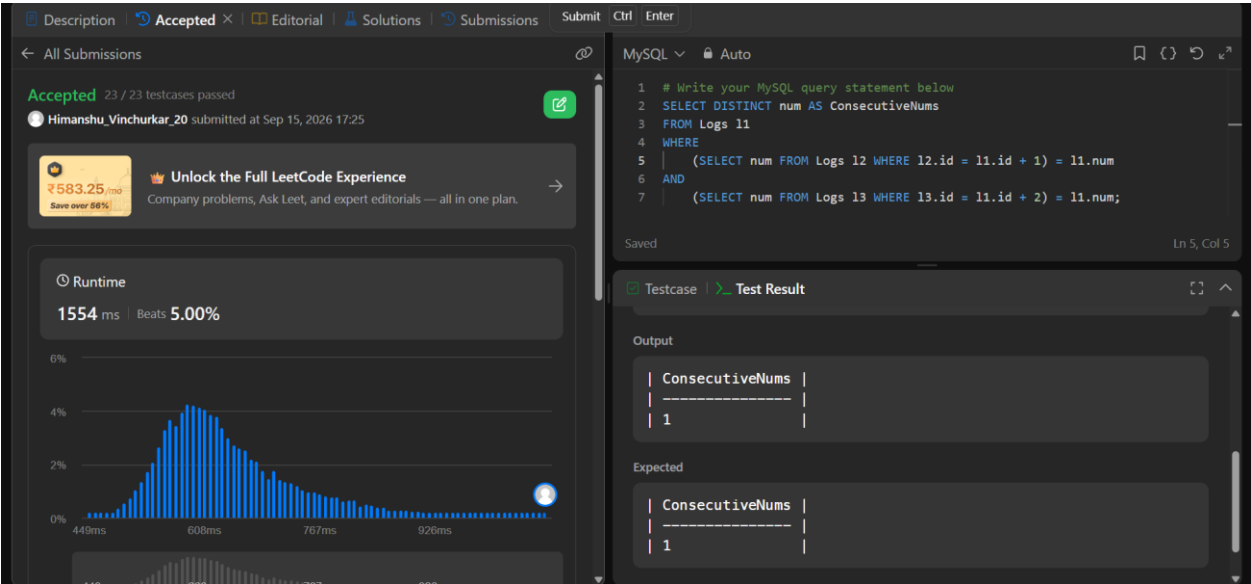
Expected

Id
2
4

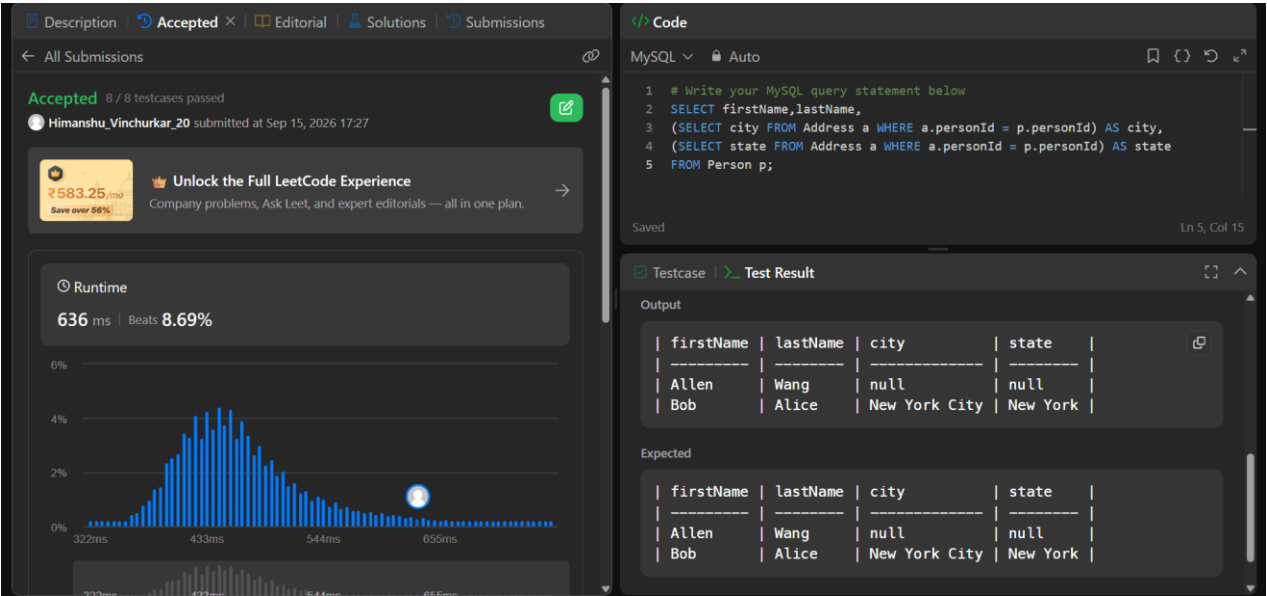
4. 607. Sales Person <https://leetcode.com/problems/sales-person/>



5. 180. Consecutive Numbers <https://leetcode.com/problems/consecutive-numbers/>



6. 175. Combine Two Tables <https://leetcode.com/problems/combine-two-tables/>



7. 178. Rank Scores <https://leetcode.com/problems/rank-scores/>

Accepted 11 / 11 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:29

₹583.25/mo Save over 56% Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime: 827 ms | Beats 7.20%

Code (MySQL):

```
1 # Write your MySQL query statement below
2 SELECT s1.score,
3 ( SELECT COUNT(DISTINCT s2.score) FROM Scores s2
4 WHERE s2.score >= s1.score) AS `rank`
5 FROM Scores s1
6 ORDER BY s1.score DESC;
```

Testcase | Test Result

Output:

score	rank
4	1
4	1
3.85	2
3.65	3
3.65	3
3.5	4

Expected:

score	rank
4	1
4	1
3.85	2
3.65	3
3.65	3
3.5	4

8. 570. Managers with at Least 5 Direct Reports <https://leetcode.com/problems/managers-with-at-least-5-direct-reports>

Accepted 11 / 11 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:31

₹583.25/mo Save over 56% Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime: 2321 ms | Beats 5.00%

Code (MySQL):

```
1 # Write your MySQL query statement below
2 SELECT name
3 FROM Employee e
4 WHERE
5 ( SELECT COUNT(*) FROM Employee e1 WHERE e1.managerId = e.id ) >= 5;
```

Testcase | Test Result

Output:

name
John

Expected:

name
John

Contribute a testcase

9. 196. Delete Duplicate Emails <https://leetcode.com/problems/delete-duplicate-emails/>

Accepted 22 / 22 testcases passed
Himanshu_Vinchurkar_20 submitted at Sep 15, 2026 17:33

₹583.25/mo Save over 56% Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime: 614 ms | Beats 94.67%

Code (MySQL):

```
1 # Write your MySQL query statement below
2 DELETE FROM Person
3 WHERE id >
4 (
5 SELECT MIN(id)
6 FROM
7 (SELECT email, MIN(id) AS id FROM Person GROUP BY email) t
8 WHERE t.email = Person.email);
```

Testcase | Test Result

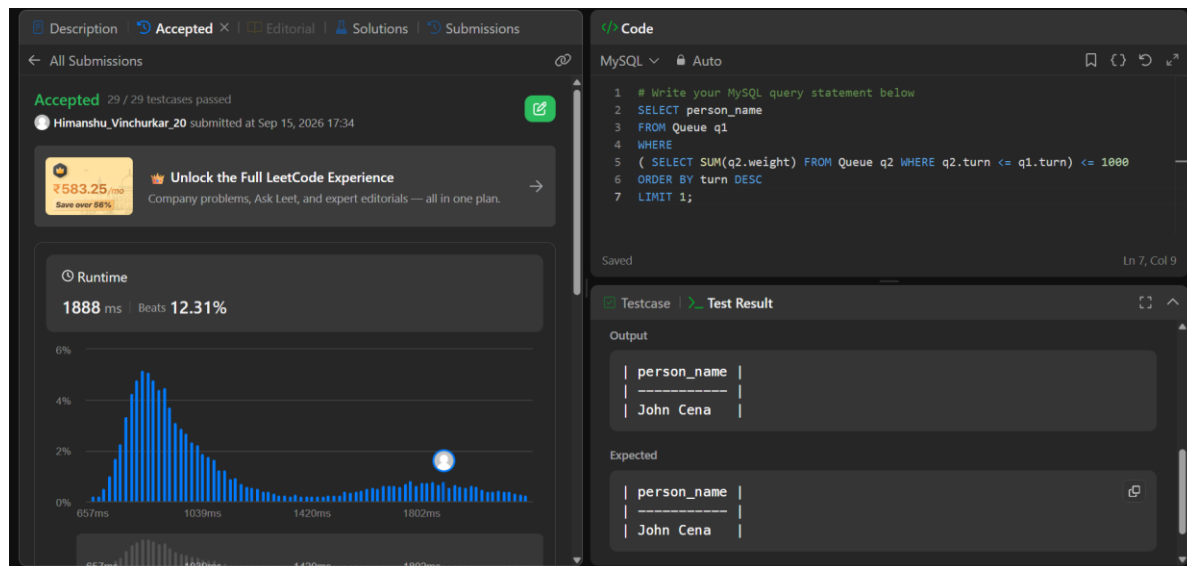
Output:

id	email
1	john@example.com
2	bob@example.com

Expected:

id	email
1	john@example.com
2	bob@example.com

10. 1204. Last Person to Fit in the Bus <https://leetcode.com/problems/last-person-to-fit-in-the-bus/>



11. 176. Second Highest Salary <https://leetcode.com/problems/second-highest-salary>

